



SDU OS Slide

Typst 幻灯片模板使用手册

API reference generated by tidy

2026-05-01

目录

项目概览	3
模块地图	3
快速开始	4
写作约定	5
标题层级	5
覆盖层	5
播放版与阅读版	5
API Reference	6
slide	6
theme	6
setup	6
overlays	6
jump	7
render-overlays	7
pause	7
meanwhile	7
foundation-theme	8
theme	8
cover	8
render-cover	8
sidebar	8
format-outline-heading	8
find-active-subheading	9
build-outline-cells	9
render-sidebar	9
draw	9
place-circle	10

项目概览

SDU OS Slide 是一个面向山东大学操作系统课程幻灯片风格的 Typst 模板.它的核心目标是保持原幻灯片的版式特征,同时提供更适合源码维护的模块结构和增量展示能力.

页边界规则

一级、二级和三级标题都是幻灯片页边界: `= Section`、`== Slide` 与 `=== Topic` 都会触发换页.三级标题不会进入右侧 sidebar,但会在新页中渲染为居中的红色标题.

模块地图

模块	职责
<code>slide.typ</code>	对外入口,导出 <code>setup</code> 、 <code>theme</code> 、 <code>pause</code> 、 <code>meanwhile</code> 、 <code>jump</code> .
<code>lib/theme.typ</code>	页面尺寸、封面、正文页框、1/2/3 级标题页边界与 <code>overlay</code> 渲染入口.
<code>lib/core/overlays.typ</code>	<code>pause</code> / <code>meanwhile</code> / <code>jump</code> 的覆盖层标记、 <code>handout</code> 与播放版渲染器.
<code>lib/foundation/theme.typ</code>	字体、颜色、页面尺寸等设计 token.
<code>lib/components/cover.typ</code>	封面页绘制.
<code>lib/components/sidebar.typ</code>	右侧目录、当前小节高亮和页内链接.
<code>lib/utils/draw.typ</code>	绘图辅助函数.

快速开始

```
#import "slide.typ": setup, theme, pause, meanwhile
```

```
#show: setup.with(  
  title: "SDU OS Slide",  
  subtitle: "Slide for SDU OS",  
  author: "arshtyi",  
  term: "2026 Spring",  
  date: datetime.today(),  
  handout: false,  
)
```

= Chapter

== First Slide

=== Optional Topic Slide

第一段内容

#pause

第二段内容

#meanwhile

这一段从第一层 overlay 就可见

`pause` 会让后续内容延迟到下一层 overlay 出现；`meanwhile` 会把后续内容重置回第一层 overlay,因此适合并行展示另一组内容.

将 `handout` 设为 `true` 可以生成阅读版 PDF: 每张幻灯片只输出最终 overlay 状态,不会把 `pause` 的中间步骤展开成多页.

写作约定

标题层级

- `=` 一级标题：章节标题,进入侧栏目录并触发新页.
- `==` 二级标题：普通幻灯片标题,进入侧栏目录并触发新页.
- `===` 三级标题：主题页标题,不进入侧栏目录,但会触发新页并居中显示.
- 更深层级标题：也会继承模板的换页规则；通常建议少用,避免目录和页边界过碎.

覆盖层

覆盖层按当前幻灯片的顶层内容顺序工作.隐藏内容使用 `hide` 保留占位,因此页面布局在不同 overlay 之间会更稳定.

```
Always visible
#pause
Visible on overlay 2
#pause
Visible on overlay 3
#meanwhile
Also visible on overlay 1
```

播放版与阅读版

```
#show: setup.with(
  title: "Lecture",
  handout: false, // presentation: keep every overlay
)
```

```
#show: setup.with(
  title: "Lecture",
  handout: true, // handout: final state only
)
```

这与 Touying 的播放/讲义分工一致：播放版保留 reveal 的节奏,阅读版保留每页最终信息密度.

API Reference

依赖于 `tidy` 生成的 API 文档。

slide

`slide.typ` 是使用者唯一需要直接导入的入口文件,它重新导出以下符号:

导出	说明
<code>setup</code>	模板主入口,通常通过 <code>#show: setup.with(..)</code> 使用.
<code>theme</code>	设计 token 字典,可在用户文档中读取字体、颜色和页面尺寸.
<code>pause</code>	进入下一层 overlay.
<code>meanwhile</code>	把后续内容重置为从第一层 overlay 可见.
<code>jump</code>	覆盖层底层跳转标记,供高级控制使用.

theme

主模板设置函数,负责把文档转换成 SDU OS slide 样式.

- `setup()`

setup

Applies the SDU OS slide theme to the document.

The setup function renders a cover page, configures the slide canvas, installs the heading styles, and enables overlay rendering through `pause`, `meanwhile`, and `jump`.

Headings always start a new page. Level 3 and deeper headings are rendered as centered in-slide titles.

- `author (str)`: Author name shown on the cover and sidebar.
- `term (str)`: Course term shown on the cover.
- `title (str)`: Presentation title shown on the cover.
- `subtitle (str)`: Subtitle shown on the cover and sidebar.
- `date (datetime)`: Date shown in the sidebar.
- `handout (bool)`: Whether to render only the final overlay state of each slide.
- `body (content)`: Presentation body.

参数

```
setup(  
  author,  
  term,  
  title,  
  subtitle,  
  date,  
  handout,  
  body  
) -> content
```

overlays

覆盖层标记与渲染逻辑,提供 Touying 风格的增量展示能力.

- `jump()`
- `render-overlays()`

变量

- `pause`
- `meanwhile`

jump

Creates an overlay jump marker.

`jump` is the common primitive behind `pause` and `meanwhile`. With `relative: true`, `n` moves the current overlay step by a relative amount. With `relative: false`, `n` sets the current overlay step absolutely.

- `n` (int): Relative or absolute overlay step.
- `relative` (bool): Whether `n` is a relative offset. Defaults to `false`.

参数

```
jump(  
  n,  
  relative  
) -> content
```

render-overlays

Renders a document body with slide overlays.

The renderer splits the body at headings and at explicit `pagebreak()` calls. In presentation mode, each resulting slide is duplicated once for each overlay step introduced by `pause`, `meanwhile`, or `jump`. In handout mode, only the final overlay state of each slide is rendered.

- `handout` (bool): Whether to render only final overlay states. Defaults to `false`.
- `body` (content): Document body to render.

参数

```
render-overlays(  
  body,  
  handout  
) -> content
```

pause content

Advances to the next overlay step.

Content after `pause` is hidden on the current overlay and appears on the next overlay. It is equivalent to `jump(1, relative: true)`.

meanwhile content

Resets subsequent content to the first overlay step.

Content after `meanwhile` appears together with the beginning of the slide, while earlier paused content keeps its own reveal timing. It is equivalent to `jump(1)`.

foundation-theme

集中存放字体、颜色和页面尺寸的设计 token.

变量

- `theme`

theme `dictionary`

Shared visual constants for the SDU OS slide theme.

The dictionary contains font families, colors, and page dimensions used by the cover, sidebar, frame, and body text.

cover

封面组件,绘制 logo、标题、徽章和装饰圆.

- `render-cover()`

render-cover

Renders the opening cover page.

The cover uses the SDU and SDU CS logos, decorative circles, and the theme's cover font and color tokens. It also resets the page counter so the first content slide starts from page 1.

- `title (str)`: Main title shown in the center of the cover.
- `subtitle (str)`: Subtitle shown below the title.
- `author (str)`: Author name shown in the lower badge.
- `term (str)`: Course term shown in the lower badge.

参数

```
render-cover(  
  title,  
  subtitle,  
  author,  
  term  
) -> content
```

sidebar

正文页右侧导航组件,生成可点击目录并高亮当前小节.

- `format-outline-heading()`
- `find-active-subheading()`
- `build-outline-cells()`
- `render-sidebar()`

format-outline-heading

Formats a level 1 or level 2 heading for the sidebar outline.

The generated heading text is linked to the original heading location. Level 1 headings use bold body text; level 2 headings keep regular body text after the numbering.

- `heading-node (content)`: Heading node returned by `query(heading)`.

参数

```
format-outline-heading(heading-node) -> content
```


find-active-subheading

Finds the active level 2 heading for a page.

A subheading is active from the page where it appears until the next outline heading starts. This keeps overlay pages inside the same highlighted sidebar row.

- outline-headings (array): Queried level 1 and level 2 headings.
- current-page (int): Current page number.

参数

```
find-active-subheading(  
    outline-headings,  
    current-page  
) -> none content
```

build-outline-cells

Builds table cells for the sidebar outline.

The returned dictionary contains the cells and the table row that should be filled with the active color.

- outline-headings (array): Queried level 1 and level 2 headings.
- active-subheading (none, content): Active level 2 heading, if any.

参数

```
build-outline-cells(  
    outline-headings,  
    active-subheading  
) -> dictionary
```

render-sidebar

Renders the right sidebar for content slides.

The sidebar shows the presentation subtitle, a linked outline built from level 1 and level 2 headings, and a footer with author and date. The active level 2 heading is highlighted according to the current page.

- title (str): Sidebar title, usually the presentation subtitle.
- author (str): Author name shown in the footer.
- date (datetime): Date shown in the footer.

参数

```
render-sidebar(  
    title,  
    author,  
    date  
) -> content
```

draw

低层绘图辅助函数.

- place-circle()

place-circle

Places a CeTZ circle by its center point.

Typst's `place` works from a top-left anchor in this template, while the visual design is easier to describe with circle centers. This helper converts center coordinates and radius to the correct placed CeTZ canvas.

- `center-x` (length): Horizontal center position.
- `center-y` (length): Vertical center position.
- `radius` (length): Circle radius.
- `style` (arguments): Additional CeTZ circle styling such as `fill` or `stroke`.

参数

```
place-circle(  
  center-x,  
  center-y,  
  radius,  
  ..style  
) -> content
```